AI-Powered Traffic Violation Detection System

UML Design Diagrams: Class, Component, Deployment and Use Case

MCA Major Project | Team NeuronX

1. About the System

The AI-Powered Traffic Violation Detection System watches live video from CCTV / IP cameras and finds traffic violations automatically. A YOLOv8 model detects vehicles and riders in each frame, the violation rules decide whether a rule was broken, and an ANPR (Automatic Number Plate Recognition) module reads the number plate. Evidence and records are saved in a PostgreSQL database, and traffic officers review violations, generate e-challans and see analytics on a React web dashboard.

Violations detected

Riding without helmet	Triple riding	Seat-belt violation
Red-light jumping	Wrong-way driving	Unauthorized movement

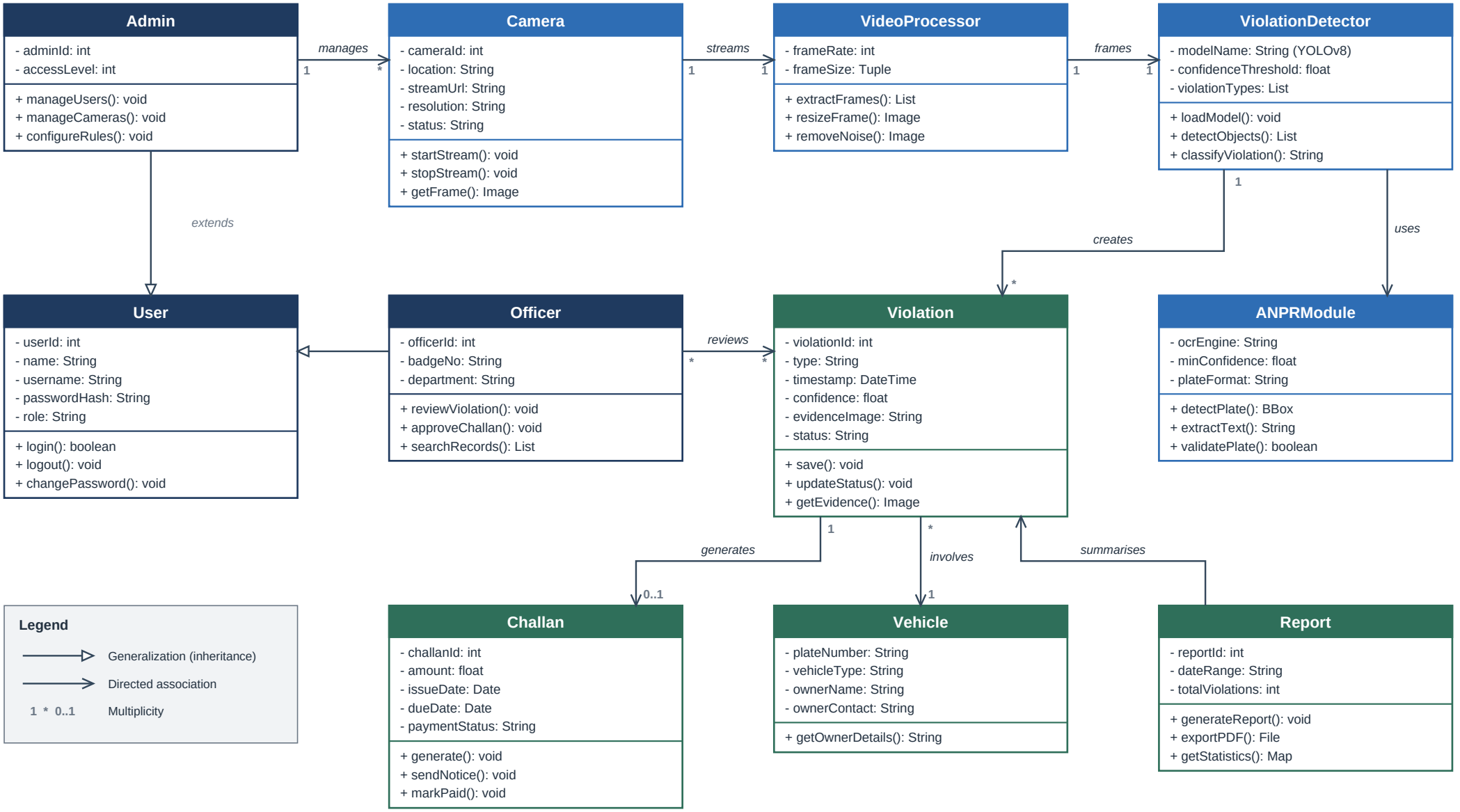
Technology stack

Layer	Technology
AI / Computer Vision	Python, YOLOv8, OpenCV, TensorFlow / PyTorch, ANPR (OCR)
Backend	Flask / FastAPI (REST API)
Frontend	React.js, React Router, Recharts
Database	PostgreSQL + file storage for evidence images
Input devices	CCTV / IP cameras (RTSP / HTTP stream)

Contents

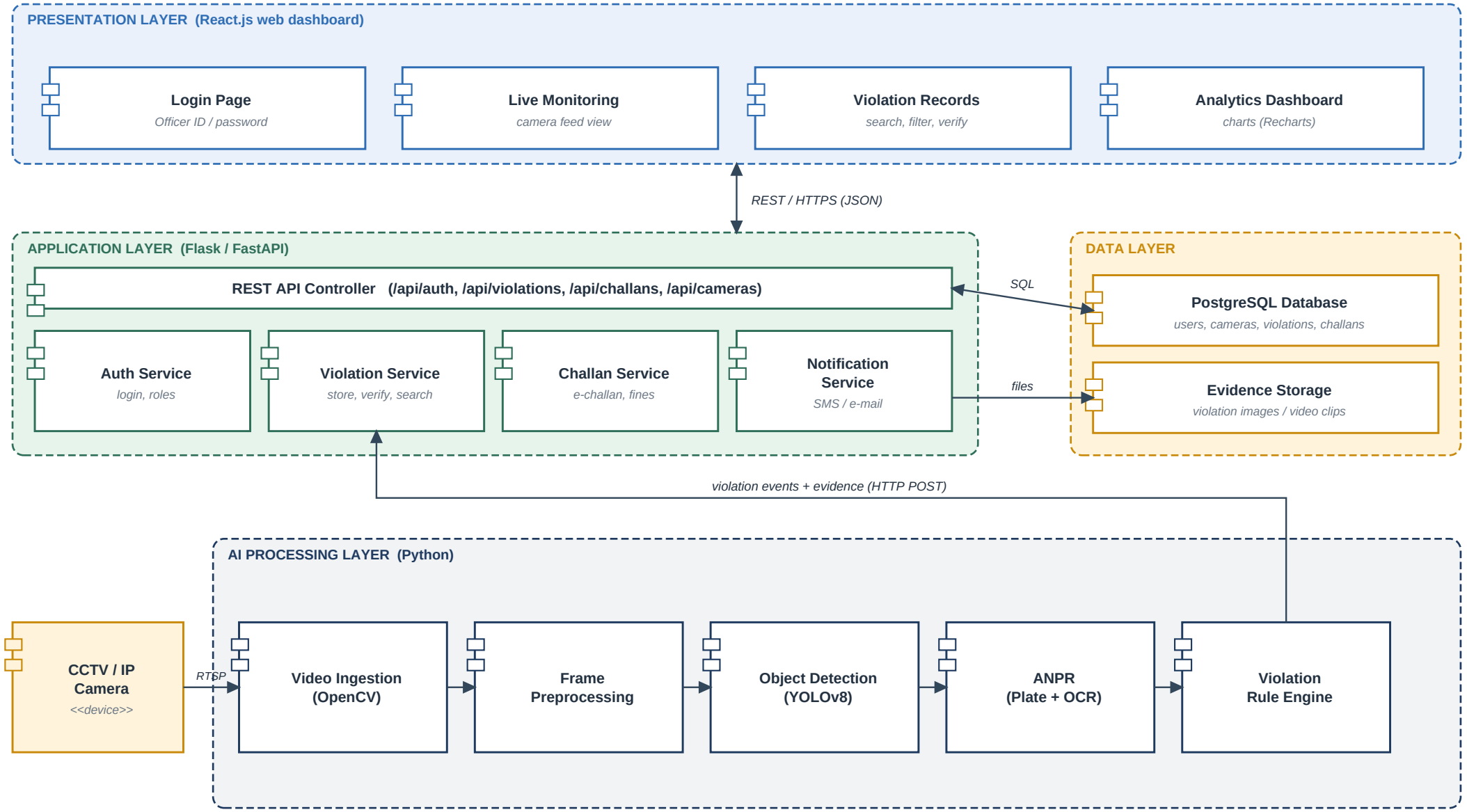
Figure	Diagram	What it shows
1	Class Diagram	Classes, their data and methods, and how they relate
2	Component Diagram	Software parts of the system and how they connect
3	Deployment Diagram	Where each part runs (hardware nodes) and the protocols used
4	Use Case Diagram	Who uses the system and what they can do
-	Explanations	Short tables that explain every diagram (last two pages)

Figure 1: Class Diagram



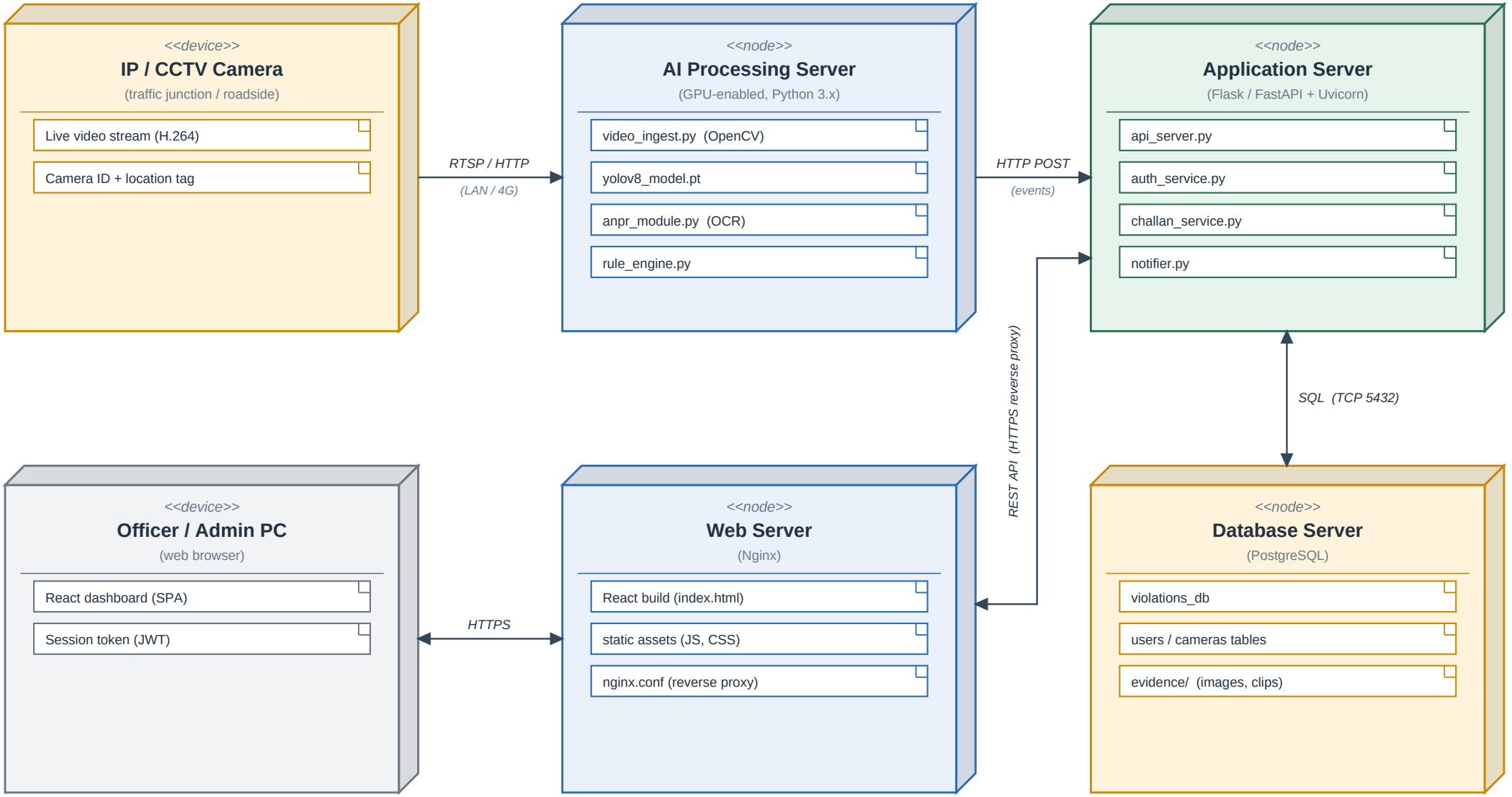
Admin and Officer inherit from User. Cameras stream video to the VideoProcessor, which feeds the ViolationDetector (YOLOv8). The detector uses the ANPRModule and creates Violation records. Each Violation involves a Vehicle and can generate a Challan.

Figure 2: Component Diagram



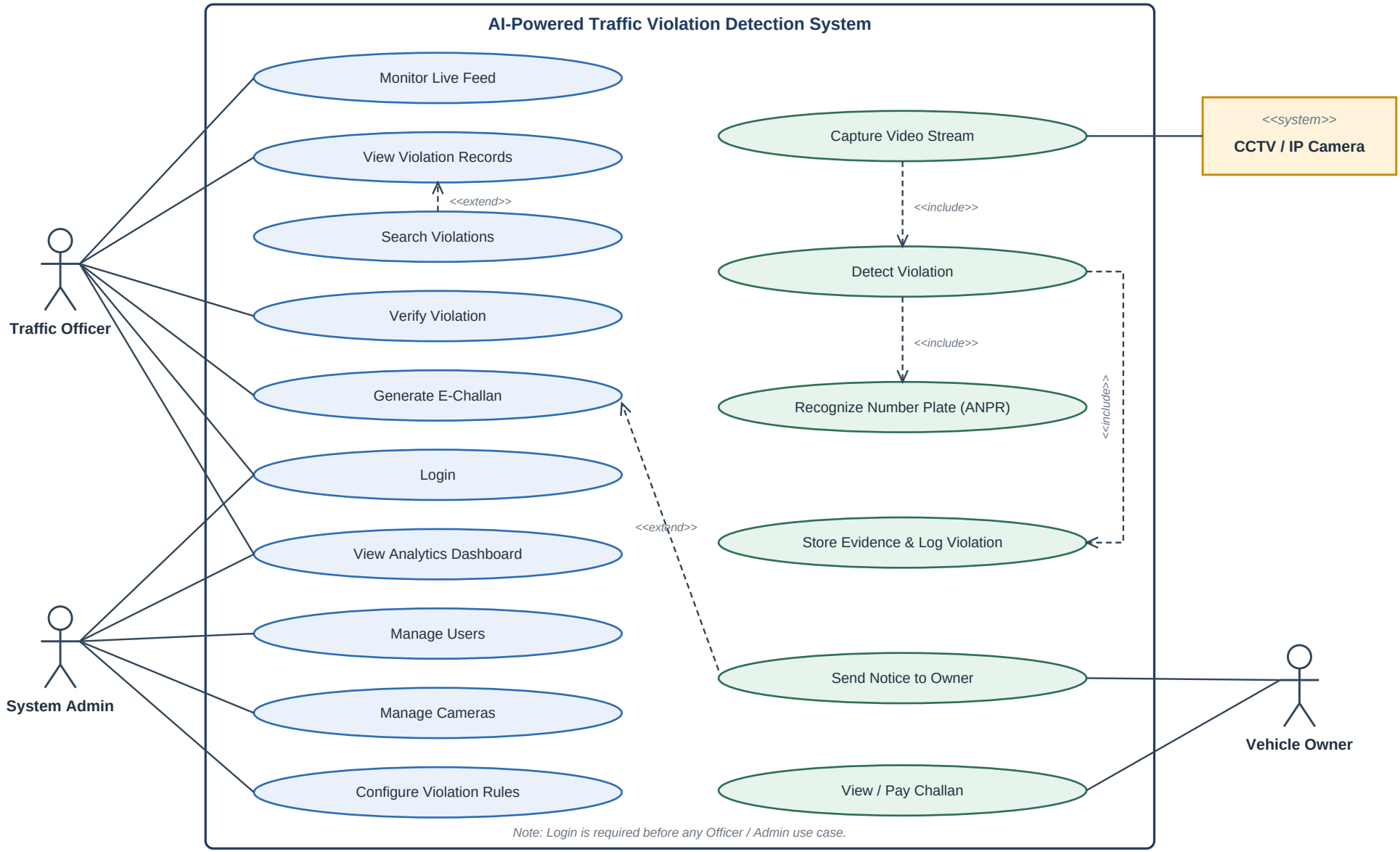
Cameras send video to the AI Processing Layer. Violation events and evidence go to the Application Layer (REST API), which stores data in PostgreSQL and evidence storage. The React dashboard talks to the REST API over HTTPS.

Figure 3: Deployment Diagram



Cameras stream to the AI Processing Server. The Application Server receives violation events and uses the Database Server. Officers open the React dashboard from their browser through the Web Server (Nginx).

Figure 4: Use Case Diagram



Traffic Officers review and act on violations, System Admins manage users, cameras and rules, cameras feed the detection process, and Vehicle Owners receive notices and pay challans.

Explanations of the Diagrams

A. Use Case Diagram

Use case	Actor	Meaning
Login	Officer, Admin	Sign in with ID / username and password; the role decides access.
Monitor Live Feed	Officer	Watch live camera streams with detected violations marked.
View Violation Records	Officer	See all violations with type, time, plate number and evidence.
Search Violations	Officer	Extends View Records: filter by plate, date, type or camera.
Verify Violation	Officer	Check the evidence and confirm or reject an AI detection.
Generate E-Challan	Officer	Create a challan for a verified violation.
View Analytics Dashboard	Officer, Admin	Charts and totals by violation type, place and time.
Manage Users / Cameras	Admin	Add, edit or disable officer accounts; register cameras and stream URLs.
Configure Violation Rules	Admin	Set confidence limits, violation types and zones (e.g. stop line).
Capture Video Stream	Camera	Sends live frames to the system.
Detect Violation	Camera (system)	YOLOv8 finds vehicles / riders and classifies the violation. Includes ANPR and Store Evidence.
Send Notice / View & Pay Challan	Vehicle Owner	Owner gets an SMS / e-mail notice and can pay the fine.

B. Class Diagram

Class	Responsibility
User, Officer, Admin	User holds login data. Officer reviews violations and approves challans. Admin manages users, cameras and rules.
Camera	Represents one CCTV / IP camera and gives video frames.
VideoProcessor	Extracts frames from the stream and cleans them (resize, remove noise).
ViolationDetector	Loads YOLOv8, detects objects and decides the violation type.
ANPRModule	Finds the number plate and reads its text.
Violation	One detected violation with time, type, confidence, evidence image and status.
Vehicle	Plate number, type and owner details.
Challan	Fine created for a violation; tracks amount, dates and payment status.
Report	Builds statistics and PDF reports from violations.

C. Component Diagram

Layer	Component	Work done
Presentation	React dashboard	Login, live monitoring, violation records and analytics for officers / admins.
Application	REST API Controller	Single entry point for all dashboard requests.
Application	Auth / Violation / Challan / Notification services	Login and roles; save and verify violations; create challans; send SMS / e-mail.
AI Processing	Video Ingestion (OpenCV)	Reads the RTSP / HTTP camera stream.
AI Processing	Frame Preprocessing	Resizes and cleans frames.
AI Processing	Object Detection (YOLOv8)	Detects vehicles, riders, helmets, seat belts and signals.
AI Processing	ANPR (Plate + OCR)	Finds the plate and reads the number.
AI Processing	Violation Rule Engine	Applies rules (helmet, triple riding, red light, wrong way) and sends events to the API.
Data	PostgreSQL, Evidence Storage	Stores users, cameras, violations, challans, and evidence images / clips.

D. Deployment Diagram

Node	Runs	Connection
IP / CCTV Camera	Live video stream	RTSP / HTTP to AI Processing Server
AI Processing Server (GPU)	OpenCV, YOLOv8 model, ANPR, rule engine	HTTP POST of violation events to Application Server
Application Server	Flask / FastAPI services	SQL over TCP 5432 to Database Server
Database Server	PostgreSQL + evidence files	Used only by the Application Server
Web Server (Nginx)	React build files, reverse proxy	HTTPS from browser; proxies /api to Application Server
Officer / Admin PC	Web browser	HTTPS to Web Server